

Step 1: Web Scraping & Local Caching

Code Explanation:

- **requests.get with Custom Headers:** Bypasses anti-bot restrictions (HTTP 403 Forbidden errors) by mimicking a real Google Chrome browser profile.
- **StringIO Wrapper:** Safely streams the raw HTML text into memory, ensuring compatibility with the latest Pandas versions and preventing **FutureWarning** deprecation alerts.
- **pd.read_html with match :** Dynamically searches the entire Wikipedia page to extract the exact table containing the phrase "Country/Territory", completely immune to unexpected HTML design changes.
- **to_csv(..., index=False) :** Saves the raw data locally to avoid making redundant network requests to Wikipedia every time the analysis script runs.

```
In [ ]: from io import StringIO
import pandas as pd
import requests

url = "https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)"

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
}

try:
    odgovor = requests.get(url, headers=headers)

    html_sadrzaj = StringIO(odgovor.text)

    tablice = pd.read_html(html_sadrzaj, match="Country/Territory")
    df = tablice[0]

    df.to_csv("sirovi_bdp.csv", index=False)
    print("USPJEŠNO UČITANO! Evo sirovih podataka:")

except Exception as e:
    print(f"Dogodila se greška: {e}")
```

USPJEŠNO UČITANO! Evo sirovih podataka:

Step 2: Data Cleaning & Statistical Analysis (Pandas & NumPy)

Code Explanation:

- **df.columns[0], df.columns[1] :** Dynamically extracts the Country column and the IMF GDP projection column to keep the dataset focused.
- **df_cisto["Drzava"] != "World" :** Filters out regional aggregates and global rows to ensure the analysis strictly reflects sovereign countries.
- **str.replace with Regex r"\[.*\]" :** Programmatically wipes out Wikipedia footnote artifacts (e.g., changing **China[n 1]** into **China**) for clean labeling.

- **pd.to_numeric with errors="coerce"** : Converts the string data into numerical floats. Any missing values or formatting anomalies (like `-N/a`) are automatically forced into official NumPy `NaN` objects.
- **dropna** : Safely discards rows with missing values to guarantee clean mathematical calculations.
- **np.sum , np.mean , np.median** : Leverages NumPy arrays for high-performance statistical calculations to evaluate global economic output, average country size, and find the true midpoint of global wealth.

```
In [ ]: import numpy as np
import pandas as pd

df = pd.read_csv("sirovi_bdp.csv")

df_cisto = df[[df.columns[0], df.columns[1]]].copy()

df_cisto.columns = ["Drzava", "BDP_Milijuni"]

df_cisto = df_cisto[df_cisto["Drzava"] != "World"]

df_cisto["Drzava"] = df_cisto["Drzava"].str.replace(r"\[.*\]", "", regex=True).s

df_cisto["BDP_Milijuni"] = pd.to_numeric(
    df_cisto["BDP_Milijuni"], errors="coerce"
)

df_cisto = df_cisto.dropna(subset=["BDP_Milijuni"])

print("\nNAKON BRISANJA:")
print(f"Preostali broj država u tablici: {len(df_cisto)}")

bdp_niz = df_cisto["BDP_Milijuni"].to_numpy()
drzave_niz = df_cisto["Drzava"].to_numpy()

ukupan_bdp = np.sum(bdp_niz)
prosjek_bdp = np.mean(bdp_niz)
medijan_bdp = np.median(bdp_niz)

print(f"Ukupan BDP svih država: {ukupan_bdp:.2f} milijuna USD")
print(f"Prosječni BDP po državi: {prosjek_bdp:.2f} milijuna USD")
print(f"Medijan BDP-a: {medijan_bdp:.2f} milijuna USD")
print("-" * 50)
```

NAKON BRISANJA:

Preostali broj država u tablici: 212

Ukupan BDP svih država: 110760477.00 milijuna USD

Prosječni BDP po državi: 522455.08 milijuna USD

Medijan BDP-a: 36904.50 milijuna USD

Step 3: Data Visualization - Top 10 Global Economies (Matplotlib)

Code Explanation:

- `df_cisto.head(10)` : Extracts the top 10 largest economies from the cleaned and sorted DataFrame for targeted visualization.
 - `ax.barh(..., [::-1])` : Creates a horizontal bar chart and uses Python slicing to reverse the order, ensuring the wealthiest nation (United States) sits perfectly at the top of the chart.
 - `ax.grid(axis="x", ...)` : Adds subtle vertical grid lines along the X-axis to make it easier for viewers to visually measure the scale of different economies.
 - `ax.text` inside the `for` loop: Programmatically places a formatted text label directly next to each bar, displaying the exact GDP value down to the dollar with proper comma formatting (e.g., \$32,383,920).
 - `plt.tight_layout()` : Automatically optimizes the canvas padding so that long country names, labels, and figures are never clipped or cut off at the margins.
- VISUALIZING

```
In [ ]: import matplotlib.pyplot as plt

top_10 = df_cisto.head(10)

fig, ax = plt.subplots(figsize=(12, 6))

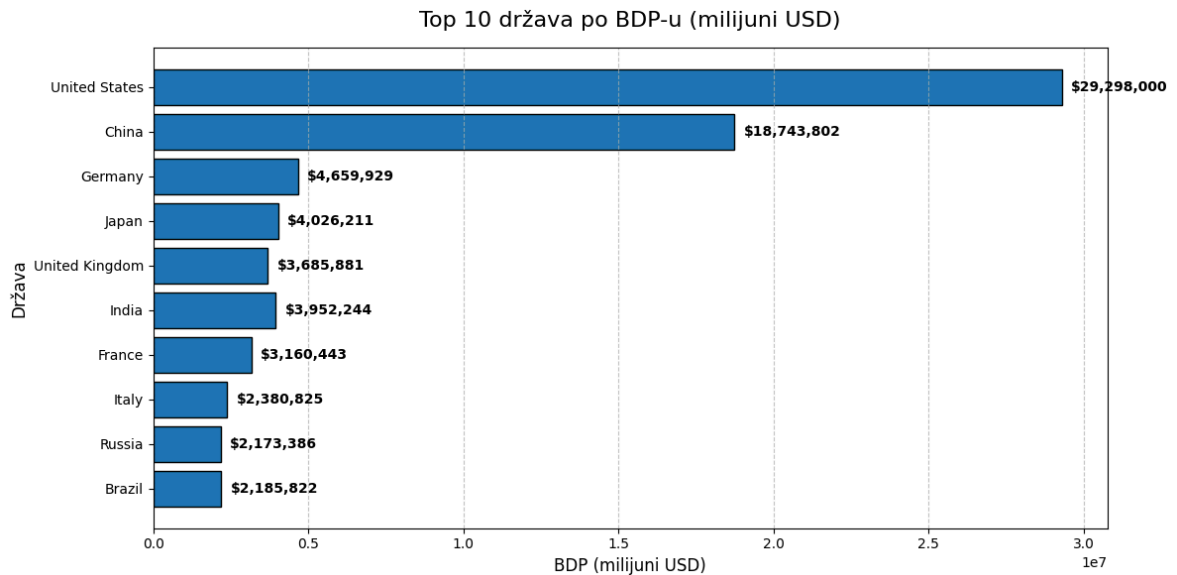
stupci = ax.barh(
    top_10["Država"][::-1],
    top_10["BDP_Milijuni"][::-1],
    color="#1f77b4",
    edgecolor="black"
)

ax.set_title("Top 10 država po BDP-u (milijuni USD)", fontsize=16, pad=15)
ax.set_xlabel("BDP (milijuni USD)", fontsize=12)
ax.set_ylabel("Država", fontsize=12)

ax.grid(axis="x", linestyle="--", alpha=0.7)

for stupac in stupci:
    sirina = stupac.get_width()
    ax.text(sirina + 300000,
            stupac.get_y() + stupac.get_height() / 2,
            f"${stupac.get_y():.0f}",
            ha="left",
            va="center",
            fontsize=10,
            fontweight="bold"
    )

plt.tight_layout()
plt.show()
```



Step 4: Data Aggregation & Global Wealth Share (Matplotlib Pie Chart)

Code Explanation:

- `df_sortirano.head(20) & df_sortirano.iloc[20:]` : Separates the top 20 economic giants from the rest of the world to keep the visualization clean and meaningful.
- `pd.concat` with "Others" row: Programmatically sums the GDP of the remaining ~190 nations into a single category labeled "Others", eliminating visual clutter.
- `explode` list comprehension: Applies a 3D-like pull-out effect (`0.05`) specifically to the top 3 global superpowers and the "Others" slice, immediately highlighting market dominance.
- `autopct` lambda function: Dynamically calculates and displays percentage slices directly inside the chart, while filtering out values below 1.5% to maintain readable text layout.
- `startangle=140 & shadow=True` : Rotates the pie for optimal alignment and introduces smooth drop-shadow aesthetics for a polished presentation.

```
In [ ]: import matplotlib.pyplot as plt

df_sortirano = df_cisto.sort_values(by="BDP_Milijuni", ascending=False)

top_20 = df_sortirano.head(20).copy()
ostatak = df_sortirano.iloc[20:]

suma_ostatak = ostatak["BDP_Milijuni"].sum()

df_pie = top_20[["Drzava", "BDP_Milijuni"]].copy()

redak_others = pd.DataFrame([{"Drzava": "Others", "BDP_Milijuni": suma_ostatak}])
df_pie = pd.concat([df_pie, redak_others], ignore_index=True)

plt.figure(figsize=(12, 12))

explode = [0.05 if i < 3 or i == 20 else 0 for i in range(21)]
```

```
plt.pie(
    df_pie["BDP_Milijuni"],
    labels=df_pie["Drzava"],
    autopct=lambda p: f"{p:.1f}%" if p > 1.5 else "",
    startangle=140,
    explode=explode,
    shadow=True,
    textprops={"fontsize": 11},
)

plt.title(
    "Share of Global GDP: Top 20 Economies vs Others", fontsize=16, pad=20
)
plt.tight_layout()
plt.show()
```

